NLP Basics in PyTorch

Tokenization · Vocabulary · Embeddings · RNN

Gianluca Barmina – gbarmina@imada.sdu.dk

01 Tokenization

02 Vocabulary

03 Embeddings

04 MLP

05 RNN

01

Tokenization

Splitting raw text into discrete units a model can process

What is Tokenization?

Definition

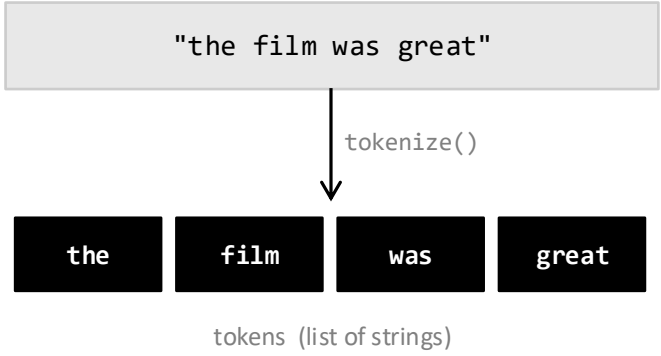
Tokenization is the process of splitting a raw string into a list of discrete units called tokens.

Why it matters

Neural models cannot operate on characters of a string directly. Every downstream component — vocabulary, embeddings, and the model — operates on the token sequence produced here.

Key properties

Vocabulary size · OOV handling · Sequence length



Strategy	Vocab	OOV?	Seq len
Whitespace	Large	Yes	Short
Character	~50	No	Long
BPE/WordPiece	Fixed	No	Medium

Tokenization — Two (Simple) Strategies

Whitespace tokenization
Split on spaces. Each word = one token.
Fast, interpretable, but OOV-prone.

Character tokenization
One token per character (spaces excluded).
Immune to OOV; produces very long sequences.

Subword (BPE / WordPiece)
Used in BERT/GPT. Best of both worlds.
(Out of scope for this exercise.)

```python
# Whitespace tokenizer
def whitespace_tokenize(text):
    return text.lower().split()

whitespace_tokenize("The film was GREAT")
# → ['the','film','was','great']

# Character tokenizer
def char_tokenize(text):
    return [ch for ch in text.lower() if ch != " "]

char_tokenize("Hi NLP")
# → ['h','i','n','l','p']
```

02

Vocabulary & Padding

Mapping tokens to integers and handling variable sequence lengths

Vocabulary — Token ↔ Integer Mapping

Purpose

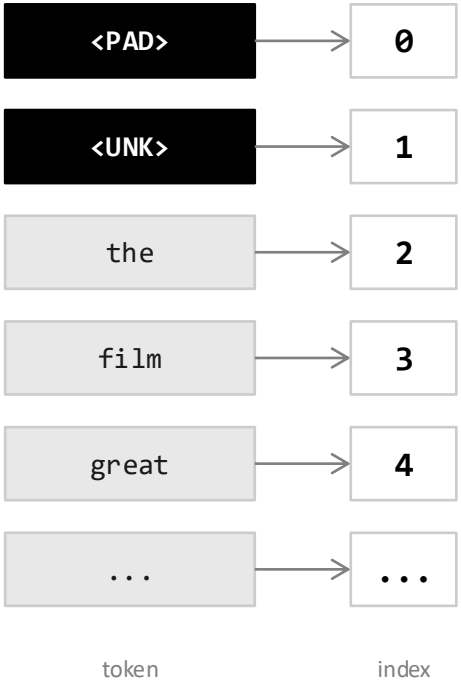
Neural networks work with numbers, not strings. A vocabulary assigns a unique integer index to every token.

Special tokens

```
<PAD>  (index 0)  — fills short sequences to a uniform length
<UNK>  (index 1)  — replaces tokens not seen at training time
```

Usage

```
build()   — scan corpus, assign indices
encode()  — tokens → indices
decode()  — indices → tokens
```



Padding — Uniform Sequence Length

Problem

GPU operations require tensors of fixed shape. Sentences in a batch have different lengths.

Solution

Right-pad shorter sequences with the <PAD> index (0) to match the longest sequence in the batch.

Result

`LongTensor of shape (B, T_max)`
`where B = batch size, T_max = max sequence length.`

```python
def pad_sequences(seqs, pad=0):
    T = max(len(s) for s in seqs)
    padded = [
        s + [pad]*(T-len(s))
        for s in seqs
    ]
    return torch.tensor(
        padded, dtype=torch.long
    )
```

03

Embeddings

Turning integer indices into dense, learnable vectors

Embeddings — From Index to Dense Vector

Why not one-hot encoding?

One-hot vectors are sparse (vocab_size wide), treat every token as equidistant, and carry no semantic information.

What is an embedding?

A learned lookup table of shape (V, D). Each row is a D-dimensional dense vector for one token. Similar tokens cluster together in embedding space.

Training

Embeddings are parameters initialised randomly and updated by gradient descent along with the rest of the model.

Embedding table (V × D)		
0	<PAD>	[0.00, 0.00, 0.00, …]
1	<UNK>	[0.31,−0.12, 0.88, …]
2	the	[0.42, 0.07,−0.55, …]
3	film	[0.81,−0.33, 0.19, …]
4	great	[0.77,−0.29, 0.23, …]
…	…	[…, …, …, …]
idx	token	← embedding vector (D dimensions) →

Embedding Layer in PyTorch

nn.Embedding

A learnable matrix of shape (vocab_size, embed_dim). Given a LongTensor of indices it returns the corresponding row vectors.

padding_idx

Setting padding_idx=0 does two things:

1. Initialises <PAD> row to all zeros
2. Zeroes the gradient for that row after every backward pass — the <PAD> vector is never updated.

Shapes

```
Input  :  (B, T)      LongTensor
Output :  (B, T, D)   FloatTensor
```

```python
import torch.nn as nn

# Create embedding layer
emb = nn.Embedding(
    num_embeddings=vocab_size,
    embedding_dim=16,
    padding_idx=0  # freeze <PAD>
)

# Forward pass
# input_ids  : (B, T)
# output     : (B, T, D)
output = emb(input_ids)

# Verify PAD is zero
assert emb(torch.tensor([0])).abs().sum() == 0
```

04

Pooling & MLP

Collapsing a sequence of vectors into a single representation

Pooling — Collapsing the Sequence Dimension

Problem

After embedding, we have a tensor of shape (B, T, D) — one vector per token. An MLP needs a fixed-size input: a single vector per sentence.

Solution: pooling

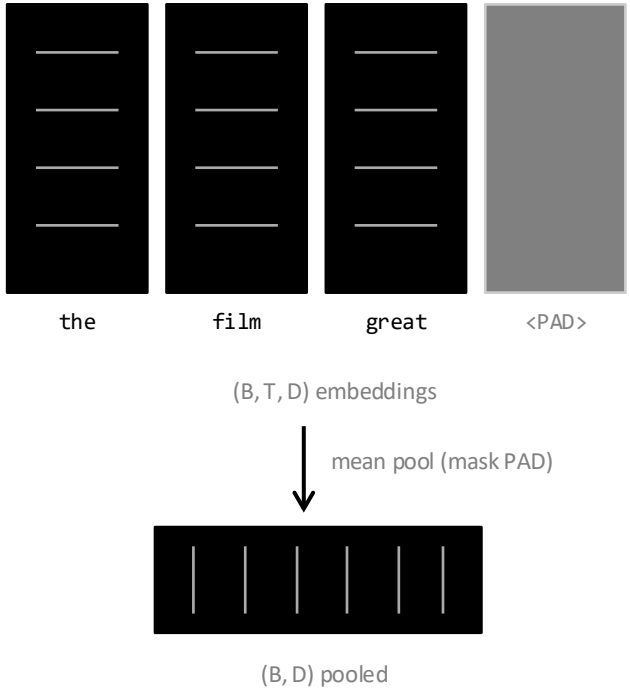
Aggregate the T token vectors into one, reducing the T dimension.

Mean pooling

Average all non-padding token vectors:

$h = (1/T_real)\ \Sigma\ e_t$

Must exclude <PAD> positions — they are zero vectors that drag the mean toward zero if counted.



Masked Mean Pooling — Implementation

Key insight

Only average over real (non-PAD) positions. Build a boolean attention_mask first.

Steps

1. Build mask — True where token ≠ <PAD>
2. Multiply embeddings by mask → zero out PAD
3. Sum over T dimension
4. Divide by count of real tokens

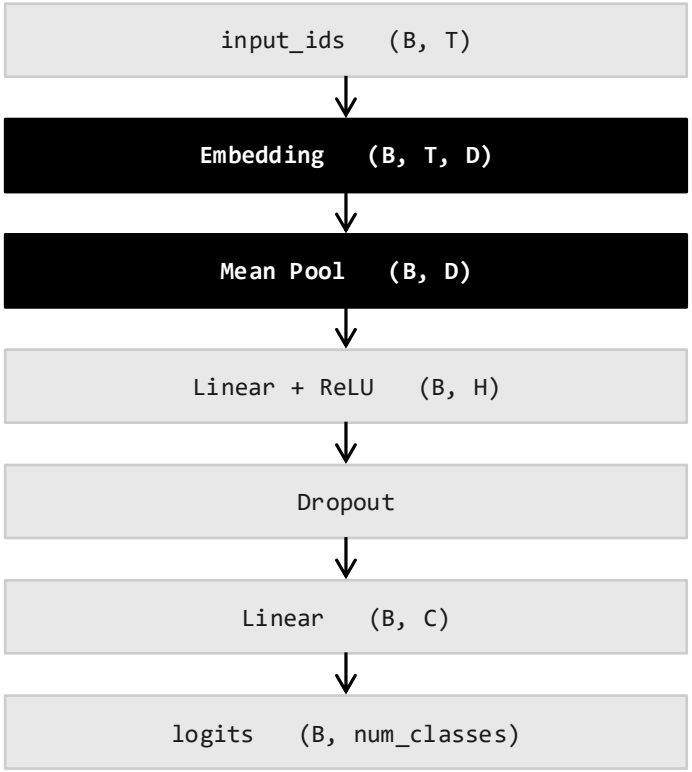
```python
def masked_mean_pool(embeddings, mask):
    # embeddings : (B, T, D)
    # mask       : (B, T)  bool

    # (B,T) → (B,T,1) broadcast
    m = mask.unsqueeze(-1).float()
```

MLP for Text Classification — Architecture

Full forward pass

```
1.  Embedding lookup     (B,T) → (B,T,D)
2.  Masked mean pooling  (B,T,D) → (B,D)
3.  Linear layer         (B,D) → (B,H)
4.  ReLU activation
5.  Dropout
6.  Linear layer         (B,H) → (B,C)
```

Output: raw logits — apply nn.CrossEntropyLoss to get probs or use logits for classification



05

Recurrent Neural Networks

Processing sequences token-by-token with a hidden state

RNNs — Sequential Processing

Core idea

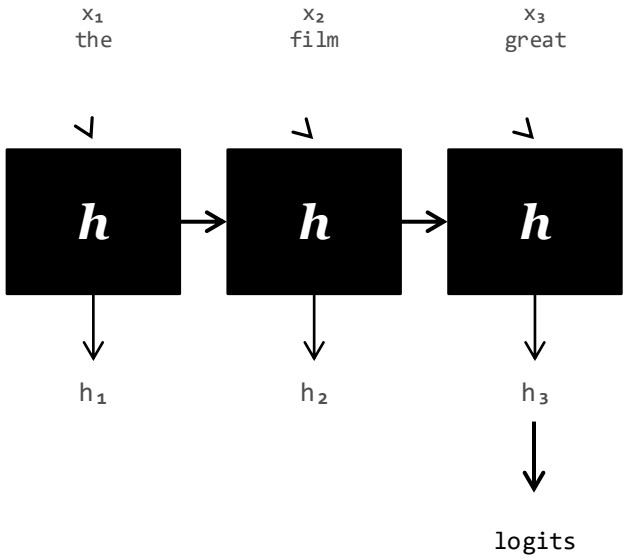
An RNN processes the token sequence one step at a time. At each step t it reads the current input x_t and its own hidden state h_{t-1} to produce a new hidden state h_t.

Recurrence formula

```
h_t = tanh( W_hh · h_{t-1}  +  W_xh · x_t  +  b )
```

Memory

h_t is a fixed-size vector that summarises all tokens seen so far. It is sensitive to word order, unlike mean pooling.



⚠ Vanilla RNNs suffer from vanishing gradients over long sequences. In practice use GRU or LSTM.

Packed Sequences — Skipping PAD Positions

Problem with padding in RNNs

Without packing, the RNN continues updating its hidden state on zero <PAD> embeddings after the real sequence ends. This corrupts h_T for shorter sequences.

pack_padded_sequence

Tells the RNN exactly how many real tokens each sequence has. The RNN only runs on real tokens and stops at the correct position.

enforce_sorted=False

PyTorch handles internal sorting automatically. No need to pre-sort the batch by length.

```
from torch.nn.utils.rnn import (
    pack_padded_sequence,
    pad_packed_sequence
)

# Real lengths (non-PAD tokens)
lengths = (input_ids != 0).sum(dim=1)

# Pack
packed = pack_padded_sequence(
    emb, lengths.cpu(),
    batch_first=True,
    enforce_sorted=False
)

# Run RNN
_, hidden = rnn(packed)
```

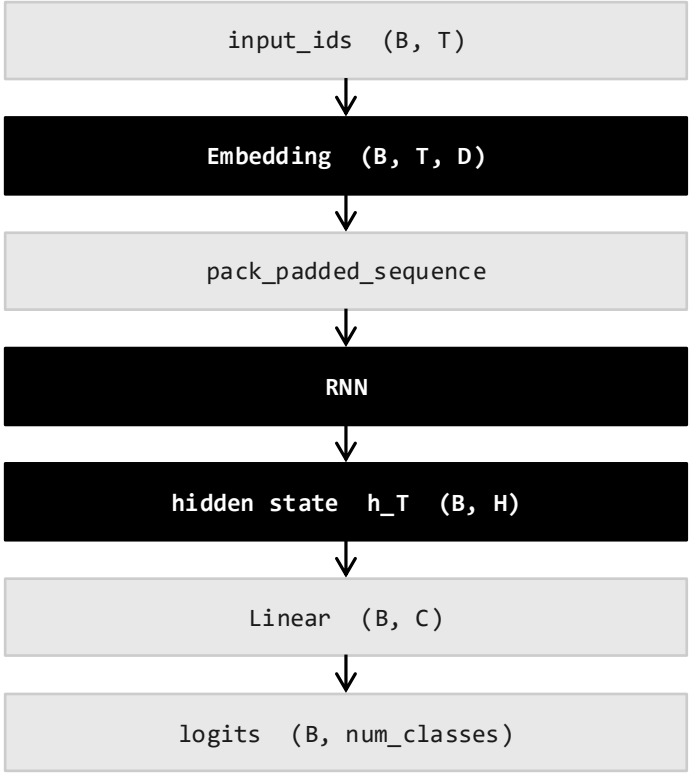
RNN for Text Classification — Architecture

Full forward pass

```
1.  Embedding lookup          (B,T) → (B,T,D)
2.  Compute real lengths      count non-PAD
3.  pack_padded_sequence      compact input
4.  RNN forward               packed → packed
5.  pad_packed_sequence       restore shape (optional, not needed)
6.  Take final hidden h_T     (B,H)
7.  Linear classifier         (B,H) → (B,C)
```

Why final hidden state?

h_T has processed every real token sequentially. Unlike mean pooling, it is sensitive to word order and can model negation.



MLP vs. RNN — When to Use Which

MLP (mean pooling)	
Word order	Ignored (bag-of-words)
Parallelism	Fully parallel — fast
OOV handling	Uses <UNK> embedding
Long-range deps	Cannot model them

RNN	
Word order	Preserved (sequential)
Parallelism	Sequential — slower
OOV handling	Uses <UNK> embedding
Long-range deps	Limited

End-to-End Pipeline Summary

1	2	3	4	5	6
Tokenize	**Vocabulary**	**Pad**	**Embed**	**Pool / RNN**	**Linear**
`text.lower().split()`	`token → int index`	`(B, T) LongTensor`	`(B, T, D) FloatTensor`	`(B, D) or (B, H)`	`(B, num_classes)`

Key concepts

`<PAD>`	Frozen zero embedding. Never contributes to gradient.	**`attention_mask`**	Boolean tensor; True = real token, False = PAD.
`<UNK>`	Fallback for tokens unseen at build() time.	**`enforce_sorted`**	Set False to let PyTorch sort the packed batch internally.
`padding_idx`	nn.Embedding arg that freezes a specific row.	**`h_T`**	RNN final hidden state — summarises the entire sequence.